

XDP-L2-Guard: Technical Reference Manual

High-Performance Layer-2 & Layer-3 Traffic Orchestration with eBPF/XDP

Krzysztof Taraszkiewicz
Józef Sztabiński

June 19, 2026

Abstract

XDP-L2-Guard is a high-performance, programmable network orchestration filter leveraging the Linux eXpress Data Path (XDP). Operating directly inside the Network Interface Card (NIC) driver hook, it provides sub-microsecond packet processing capabilities for Layer-2 and Layer-3 traffic. This enables wire-speed dropping, packet redirection, hairpinning, and state-aware Network Address Translation (NAT) before the Linux kernel allocates socket buffers (`sk_buff`). This manual serves as an exhaustive, low-level technical reference covering C-level memory structures, Python CO-RE management, checksum algorithms, and deployment strategies.

Contents

1	eBPF vs. Traditional Linux Kernel Modules	1
1.1	Method of Operation and Privilege Acquisition	1
1.2	Key Benefits of eBPF over LKMs	2
2	System Architecture and Paradigm	2
2.1	Architectural Component Diagram	3
3	Data Plane Implementation (eBPF/XDP)	3
3.1	Memory Layout and Pointer Validation	3
3.2	Action Map Schema	4
3.3	Low-Level Action Semantics	4
3.3.1	XDP_DROP	4
3.3.2	XDP_TX	4
3.3.3	XDP_REDIRECT & ACTION_NAT (DNAT)	4
4	Control Plane: Orchestration and Polling	5
4.1	The Loader Engine (<code>loader.py</code>)	5
4.2	Management CLI (<code>guard.py</code> & <code>guard.sh</code>)	5
5	Action Showcase and Testing Examples	6
5.1	Network Topology Setup	6
5.2	Applying Actions	6
6	Operational DevOps & Diagnostics	6
6.1	Performance Tuning & Real-World Limits	6
6.1.1	Case Study: Intel NIC Native Mode Testing	6
6.2	Diagnostic Matrix	7

7	Exploring Carrier Ethernet Potential via XDP	7
7.1	Testing Internal Routing Capabilities	7
7.2	Hardware Offloading and Intel NICs	8
8	Testing the Software Switch Concept	8
8.1	eBPF as an Alternative to iptables	8
8.2	Basic NAT and the Data Plane Potential	8
9	Project Summary and Conclusions	8
9.1	Impressions and Lessons Learned	8
9.2	Why This Project Was Worth Doing	9
9.3	Future Research and Testing Directions	9

1 eBPF vs. Traditional Linux Kernel Modules

Historically, modifying or extending the behavior of the Linux kernel required writing a Linux Kernel Module (LKM) or recompiling the kernel source code. While LKMs have full privileges and unrestricted access to kernel memory and execution flows, they inherently introduce significant risks. A bug in an LKM, such as a null pointer dereference or an infinite loop, can instantly lead to a kernel panic, crashing the entire operating system. Furthermore, LKMs pose security risks as they run with unrestricted Ring 0 privileges without built-in isolation.

eBPF (Extended Berkeley Packet Filter) introduces a fundamentally different operational paradigm. It allows developers to safely execute custom bytecode within the kernel without changing kernel source code or loading standard modules.

1.1 Method of Operation and Privilege Acquisition

Unlike a traditional kernel module that is linked directly against kernel symbols and executed natively as kernel code, eBPF programs are injected into the kernel via the `bpf()` system call from user-space. However, before any eBPF program is allowed to execute, it must pass through a strict **Kernel Verifier**.

The Verifier performs static analysis on the eBPF bytecode to guarantee that the program is safe to run. It ensures that:

- There are no infinite loops (all programs must terminate).
- All memory accesses are strictly bounds-checked.
- No uninitialized variables are used.
- The program size does not exceed strict complexity limits.

Once the Verifier approves the bytecode, the eBPF **JIT (Just-In-Time) Compiler** translates the bytecode into native machine instructions (e.g., x86_64 or ARM64) specific to the host hardware. This JIT-compiled code then runs at near-native speed directly in kernel space.

Thus, eBPF obtains kernel privileges not by bypassing security layers, but by mathematically proving to the kernel that its execution is safe. It cannot access arbitrary kernel memory; it is strictly limited to context-specific data structures (like the `xdp_md` struct for packet data) and predefined kernel helper functions (`bpf_helpers`) that provide a safe API to interact with the broader system.

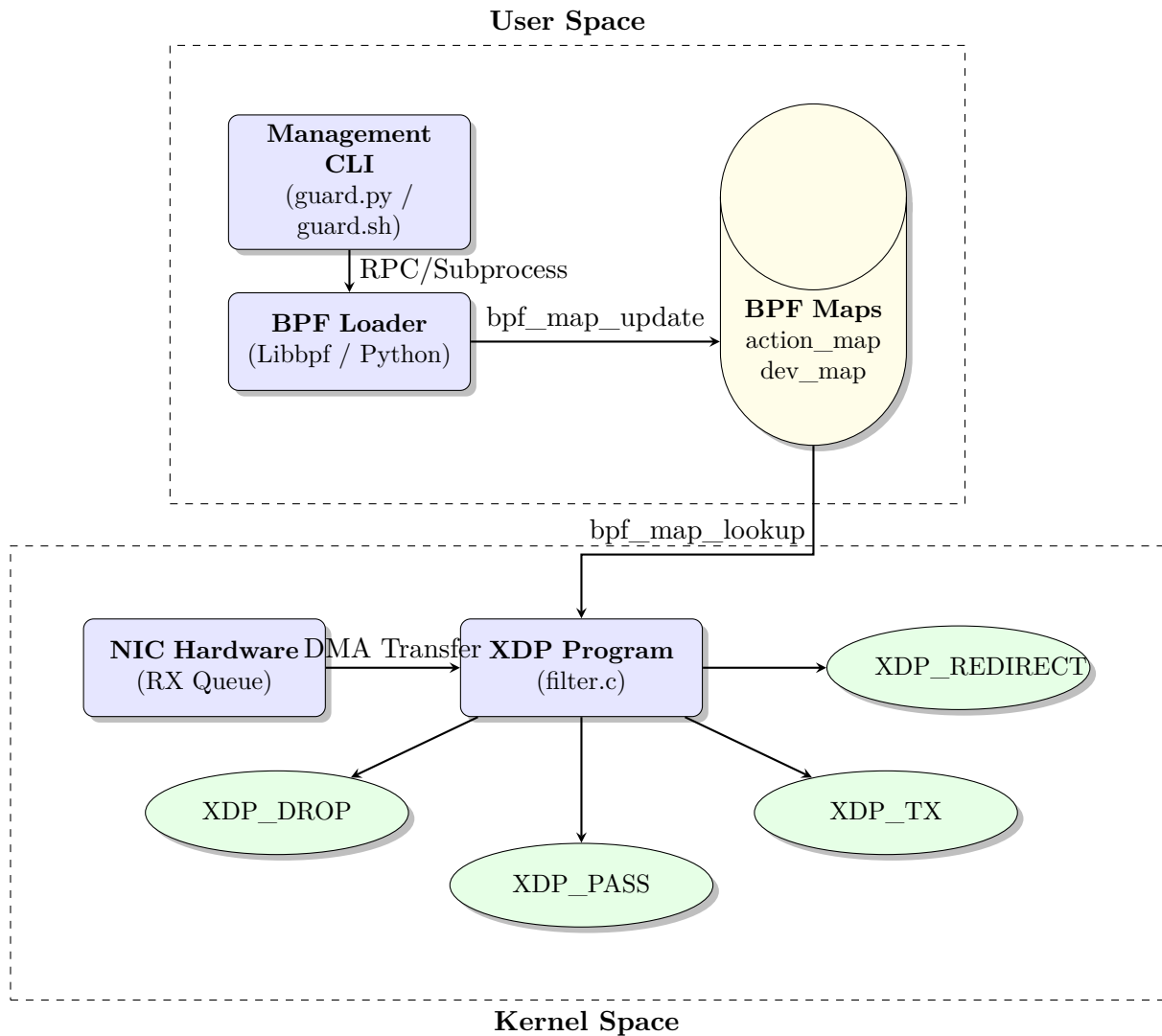
1.2 Key Benefits of eBPF over LKMs

- **Safety and Stability:** The rigorous verification process ensures that an eBPF program cannot crash the system. If the code violates memory boundaries or takes too long to execute, the Verifier simply rejects it at load time.
- **Dynamic Loading and Updates:** eBPF programs can be loaded, updated, and unloaded on the fly without system reboots or network interruptions. This enables dynamic and seamless updates to packet filtering rules.
- **High Performance:** Thanks to JIT compilation, eBPF programs run at native speeds. In the context of XDP, this allows packet processing to occur directly in the NIC driver before the kernel even allocates memory (`sk_buff`), yielding sub-microsecond latency.
- **Security and Isolation:** Because eBPF is constrained to specific hooks and helpers, it dramatically reduces the attack surface compared to an LKM that has absolute control over the system.

2 System Architecture and Paradigm

The system is strictly architected around the paradigm of control/data plane separation. The **Data Plane** is written in restricted C, compiled via Clang/LLVM into eBPF bytecode, and executes securely within the kernel context. The **Control Plane** utilizes Python 3 to orchestrate the lifecycle of the eBPF program, utilizing `iproute2` and `bpftool` to manage asynchronous state updates.

2.1 Architectural Component Diagram



3 Data Plane Implementation (eBPF/XDP)

The data plane code (`src/data_plane/filter.c`) hooks into the NIC's NAPI poll loop via the `SEC("xdp")` macro. The function `xdp_drop_logic` acts as the primary ingress orchestrator.

3.1 Memory Layout and Pointer Validation

eBPF programs operate in a verified environment. Before the verifier allows access to packet memory, the code must prove that the memory bounds are safe. The program receives a `struct xdp_md` context, providing DMA mapped pointers:

```
void *data = (void *) (long) ctx->data;
void *data_end = (void *) (long) ctx->data_end;
```

We utilize a custom `BOUNDS_CHECK` macro defined in `headers.h`:

```
#define BOUNDS_CHECK(pointer, type, data_end) \
do { \
    if ((void *) (pointer) + sizeof(type) > (void *) (data_end)) \
        return XDP_PASS; \
}
```

```
} while(0)
```

The parsing pipeline proceeds strictly layer by layer:

1. **Layer 2 (Ethernet)**: Cast data to `struct ethhdr`. Verify bounds. Check if `eth->h_proto == bpf_htons(ETH_P_IP)`. If not, bypass to kernel via `XDP_PASS`.
2. **Layer 3 (IPv4)**: Cast data + `sizeof(struct ethhdr)` to `struct iphdr`. Verify bounds. Extract the Destination IP (`ip->daddr`).

3.2 Action Map Schema

The extracted IP is used as a lookup key in the primary BPF hash map, `action_map`. The map expects an IPv4 address in network byte order (`__u32`) and returns a `struct action_cfg`:

```
struct action_cfg {
    __u32 target;           // 0: PASS, 1: DROP, 2: TX, 3: REDIRECT, 4:
    NAT
    __u32 new_ip;           // Destination IP for NAT scenarios
    __u32 ifindex;          // OS Interface index for REDIRECT
    __u64 dropped_packets;  // Atomic performance counter
};
```

3.3 Low-Level Action Semantics

Depending on the retrieved `target`, the kernel executes one of the following routines:

3.3.1 XDP_DROP

The simplest and fastest action. The NIC driver drops the frame and immediately recycles the DMA buffer. The code atomically increments the global array map `total_dropped` and the rule-specific `dropped_packets`.

3.3.2 XDP_TX

`XDP_TX` bounces the packet back out of the interface it arrived on. In our implementation, this is used for advanced ICMP manipulation. If an `ICMP_ECHO` (Ping) is detected:

1. The type is rewritten to `ICMP_ECHOREPLY` (0).
2. **Checksum Math**: Because Type 8 changed to Type 0, the 16-bit 1's complement checksum must be incrementally updated by adding `0x0800`.
3. MAC and IP addresses are swapped utilizing `__builtin_memcpy` for optimal inline compilation.

3.3.3 XDP_REDIRECT & ACTION_NAT (DNAT)

`ACTION_NAT` performs Destination Network Address Translation directly in hardware.

```
__u32 old_daddr = ip->daddr;
__u32 new_daddr = cfg->new_ip;
__u32 csum = ~bpf_ntohs(ip->check) & 0xFFFF;
csum += (~old_daddr & 0xFFFF) & 0xFFFF + (new_daddr & 0xFFFF);
csum += (~old_daddr >> 16) & 0xFFFF + (new_daddr >> 16);
while (csum >> 16) csum = (csum & 0xFFFF) + (csum >> 16);
ip->check = bpf_htons(~csum & 0xFFFF);
```

This bitwise logic incrementally updates the IP header checksum without requiring a full packet recalculation, ensuring sub-microsecond latency. `XDP_REDIRECT` follows the exact same logic but concludes by forwarding the frame to an interface indexed in `dev_map`.

4 Control Plane: Orchestration and Polling

The Control Plane comprises two primary Python scripts: `loader.py` for lifecycle management and `guard.py` for CLI rule definition.

4.1 The Loader Engine (`loader.py`)

The loader handles Ahead-Of-Time (AOT) compilation and kernel attachment via the `iproute2` toolchain.

- **Compilation:** Triggers `make` to compile `filter.c` into the ELF object `filter.o`.
- **Mode Selection:** Detects whether to mount in high-performance native mode (`xdpdrv`) or generic software mode (`xdpgeneric`).
- **Attachment:** Uses `ip -force link set dev <interface> xdpdrv obj filter.o sec xdp`. The `-force` flag guarantees atomic overwrites of zombie programs.
- **Asynchronous Polling:** Operates an infinite loop (`time.sleep(1)`) that invokes `bpftool -j map dump name action_map`. It decodes the JSON payload, unpacks the byte streams, and translates the raw metrics into human-readable logs.

4.2 Management CLI (`guard.py` & `guard.sh`)

The project provides two identical `iptables`-like interfaces to manipulate the active BPF maps via `bpftool`:

- **`src/control_plane/guard.py`:** The primary Python implementation using `argparse`.
- **`scripts/guard.sh`:** A pure Bash alternative utilizing `awk` and `sed` for environments where Python is unavailable or undesired.

Both scripts share the exact same syntax and arguments.

CLI Syntax Examples:

```
# Append a standard DROP rule for a malicious actor (Python version)
sudo python3 guard.py --append --destination 192.168.1.50 --jump DROP

# Establish a hardware-level NAT translation (Bash version)
sudo ./guard.sh -A -d 10.0.0.100 -j NAT --to-destination 10.0.0.250

# Redirect traffic arriving for 10.0.0.5 to another Virtual Ethernet
sudo ./guard.sh -A -d 10.0.0.5 -j REDIRECT --oif veth1

# List all active rules dynamically pulled from kernel memory
sudo python3 guard.py --list
```

Internally, both scripts resolve system interfaces (e.g., `veth1`) to integer `ifindex` values via `/sys/class/net/<iface>/ifindex` and format the C-struct byte arrays (either via Python's `struct.pack` or Bash's `printf`) to be injected into the kernel via `bpftool map update`.

5 Action Showcase and Testing Examples

To demonstrate and validate the capabilities of XDP-L2-Guard, the `scripts/example/` directory provides an interactive playground utilizing Linux Network Namespaces (netns).

5.1 Network Topology Setup

The `setup.sh` script initializes a simulated 3-node network environment:

- **ns1 (Filter/Router):** Assigned IP 10.0.0.1. The eBPF object is attached to virtual interfaces `v1-2` and `v1-3`. IPv4 forwarding is enabled.
- **ns2 (Sender):** Assigned IP 10.0.0.2. Acts as the source of traffic.
- **ns3 (Target):** Assigned IP 10.0.0.3 in promiscuous mode to observe redirected packets.

5.2 Applying Actions

The `set_action.sh` script uses `guard.sh` to dynamically apply rules for traffic originating from `ns2`:

```
# Terminal 1: Run the setup and keep the XDP filter active
sudo ./setup.sh

# Terminal 2: Test different XDP actions in real-time
./set_action.sh drop      # Traffic from ns2 is silently dropped
./set_action.sh redirect  # Traffic from ns2 is bounced to ns3
./set_action.sh tx        # Hairpin ICMP echo (Ping responds instantly)
./set_action.sh nat       # Destination IP is rewritten to 10.0.0.3
```

This setup proves that complex Layer-2 (REDIRECT) and Layer-3 (NAT) operations occur entirely within the eBPF layer, completely bypassing the conventional routing logic in `ns1`.

6 Operational DevOps & Diagnostics

6.1 Performance Tuning & Real-World Limits

To achieve theoretical maximum throughput (10-40 Gbps), several hardware-level optimizations are mandatory.

- **Native Mode (`xdpdrv`) vs. Generic Mode (`xdpgeneric`):** Native mode runs the eBPF program directly inside the NIC driver, before the kernel allocates any memory. Generic mode runs later in the stack. Native mode is essential for flood protection.
- **Disable Hardware Offloads:** Offloads like Generic Receive Offload (GRO) aggregate packets before XDP sees them, bypassing the filter. Disable via: `sudo ethtool -K eth0 gro off gso off rxvlan off`.
- **CPU Pinning:** Ensure XDP IRQs are pinned to dedicated physical cores using `smp_affinity`.

6.1.1 Case Study: Intel NIC Native Mode Testing

During real-world load testing utilizing multiple independent attacking machines flooding an Intel network interface, our goal was to leverage Native XDP mode (`xdpdrv`) for maximum hardware acceleration. Ultimately, we encountered low-level hardware configuration challenges rather than hitting the computational limits of the host CPU.

While we were unable to perfectly align the NIC driver configuration to achieve wire-speed processing in this specific test, we recognize this as a critical area for future development. We are fully aware of the massive performance potential that native hardware execution offers. We were exceptionally close to a working native implementation, and resolving these hardware-specific configuration nuances remains a high-priority milestone for the next evolution of XDP-L2-Guard.

6.2 Diagnostic Matrix

Symptom	Root Cause Analysis	Remediation
<code>libbpf: load fail</code>	eBPF Verifier rejected code bounds.	Inspect <code>dmesg</code> . Ensure <code>BOUNDS_CHECK</code> covers all accessed memory.
Rules applied, no drop	Attached to wrong interface or mode.	Check with <code>ip link show</code> . Ensure XDP is attached to the physical interface, not a bridge.
EBUSY during attach	Another XDP program is already loaded.	Use the <code>-force</code> flag in <code>ip link set dev <iface> xdp obj ...</code>
High SoftIRQ CPU	Lock contention on <code>total_dropped</code> .	Refactor map to <code>BPF_MAP_TYPE_PERCPU_ARRAY</code> .
Packets bypass filter	Hardware offloading is active.	Disable GRO/LRO using <code>ethtool -K <iface> gro off</code> .
BPF map creation fails	<code>RLIMIT_MEMLOCK</code> is too low.	Increase memory limits: <code>ulimit -l unlimited</code> .
<code>XDP_TX</code> fails silently	Checksum recalculation is incorrect.	Verify 1's complement math. Ensure MAC swapping logic is correct.

7 Exploring Carrier Ethernet Potential via XDP

While a standard Linux distribution is not typically used as a core Carrier Ethernet router, modern kernel capabilities provide the building blocks to test these concepts. Carrier Ethernet relies on high-throughput packet delivery and internal labeling mechanisms (like MPLS or VLANs) to route traffic effectively. Our project, XDP-L2-Guard, serves as a small-scale testing ground to understand the potential of eBPF and XDP in these scenarios.

7.1 Testing Internal Routing Capabilities

Historically, routing based on MPLS labels in Linux relies on the standard kernel network stack (via modules like `mpls_router`), which introduces CPU overhead. Our implementation in `filter.c` currently focuses on basic IPv4 destination lookups, but it effectively demonstrates the underlying principle required for a Label Switch Router (LSR).

By operating at the XDP hook, we intercept packets before they reach the kernel's `sk_buff` allocation. The `XDP_REDIRECT` functionality we tested shows that it is entirely possible to

construct a fast-path data plane. While we have not implemented full MPLS parsing, our code proves the potential: eBPF logic could easily be extended to read MPLS labels directly from the `data` pointers and route packets instantly using a `dev_map`.

7.2 Hardware Offloading and Intel NICs

We tested XDP-L2-Guard with the goal of understanding Native XDP Mode (`xdpdrv`) on modern Intel Network Interface Cards (such as the 700 and 800 ice series). In theory, running eBPF bytecode directly inside the NIC's RX queue driver offers performance comparable to DPDK (Data Plane Development Kit, an industry-standard framework that achieves high speeds by completely bypassing the OS kernel to process packets in user space), without leaving the Linux kernel environment.

Although our project is merely a proof-of-concept, it highlights how hardware acceleration (like Intel's Flow Director) combined with XDP can intelligently distribute and process traffic at extremely low latency. Our experiments confirm the theoretical potential of using Linux and Intel hardware to process Carrier-grade traffic.

8 Testing the Software Switch Concept

One of the primary goals of XDP-L2-Guard was to explore how eBPF can serve as a foundation for a programmable software switch, and how it compares to traditional firewalling tools.

8.1 eBPF as an Alternative to iptables

Traditionally, packet filtering and NAT on Linux rely on `iptables`. However, `iptables` uses a sequential list model ($O(n)$ complexity), meaning performance degrades as the rule count grows. Our project allowed us to practically test the performance benefits of eBPF maps.

By implementing an `action_map` (`BPF_MAP_TYPE_HASH`), our XDP script achieves constant-time $O(1)$ lookups. Although our Python control plane (`guard.py`) only manages a fraction of the rules a real-world enterprise might use, the fundamental mechanism we demonstrated scales effortlessly, proving the viability of eBPF over legacy `iptables`.

8.2 Basic NAT and the Data Plane Potential

Instead of relying on the kernel's netfilter stack, we implemented a basic `ACTION_NAT` in C to test direct packet manipulation. Our code successfully reads destination IPs, rewrites them, and recalculates the 1's complement checksum using bitwise math directly inside `filter.c`.

When combined with `XDP_REDIRECT`, this creates a rudimentary but high-speed software switch. While XDP-L2-Guard is not a complete, production-ready router, it successfully demonstrates a tiny fraction of eBPF's capabilities, confirming that Linux has the architectural potential to handle complex, high-speed routing and labeling at the lowest possible software level.

9 Project Summary and Conclusions

9.1 Impressions and Lessons Learned

Executing the XDP-L2-Guard project provided invaluable hands-on experience with the lowest levels of the Linux networking stack. Our primary impression is one of profound capability mixed with steep learning curves. eBPF and XDP offer unprecedented power to intercept and modify packets at wire speed, but this power comes at the cost of strict Verifier constraints and complex C-level memory handling. We successfully proved that we could write a functional

data plane capable of parsing IP headers, rewriting checksums, and orchestrating basic NAT or redirects—all before the kernel allocates any standard socket buffers.

9.2 Why This Project Was Worth Doing

This project was highly valuable because it demystified the mechanisms behind modern cloud-native networking and software-defined networking (SDN). By building XDP-L2-Guard from scratch, we shifted our understanding of Linux from a simple endpoint or basic firewall into a highly programmable, hardware-adjacent switch. It allowed us to practically experience the limitations of legacy tools like `iptables` and fully grasp why the industry is rapidly adopting eBPF. The knowledge gained regarding Native XDP, memory bounds-checking, and control/-data plane separation forms a critical foundation for modern network engineering.

9.3 Future Research and Testing Directions

While XDP-L2-Guard serves as a successful proof-of-concept, there are numerous avenues for potential future exploration:

- **Deep Layer-2/Label Parsing:** Expanding `filter.c` to natively parse, push, and pop MPLS labels or VLAN tags to simulate a complete Carrier Ethernet Label Switch Router (LSR).
- **Advanced Hardware Tuning:** Further resolving the low-level hardware configuration nuances of Intel NICs to fully unleash Native Mode (`xdpdrv`) and benchmark it against DPDK-based solutions at 40+ Gbps line rates.
- **Stateful Tracking:** Transitioning from stateless DNAT to fully stateful connections by implementing connection tracking (`conntrack`) utilizing eBPF LRU (Least Recently Used) hash maps.
- **IPv6 Support:** Extending the C codebase and bounds-checking logic to comprehensively handle IPv6 headers and ICMPv6 routing protocols.